

Instituto de Informática
Departamento de Informática Aplicada

Dados de identificação**Período Letivo:** 2026/2**Professor Responsável:** Karina Kohl Silveira**Disciplina:** TESTE E VERIFICAÇÃO DE SOFTWARE**Sigla:** INF01088**Créditos:** 2**Carga Horária**

CH Teórica: 20h CH Prática: 10h Total: 30h

CH Coletiva: 20h CH Autônoma: 10h CH Individual: 0h

Carga Horária de prática Extensionista (CHE) 0h

Súmula

Introdução à verificação e validação de software. Requisitos de software. Fundamentos de teste de software. Fundamentos de verificação formal de software.

Currículos

Currículos	Etapla Aconselhada	Natureza
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO	2	Obrigatória

Objetivos

A disciplina tem como objetivo introduzir os estudantes aos fundamentos de testes e verificação de software, desenvolvendo uma compreensão inicial e prática do pensamento orientado a testes. Busca-se capacitar o aluno a interpretar requisitos e regras de negócio simples, transformando-os em cenários e casos de teste que permitam avaliar o comportamento esperado de pequenos programas. Ao longo do semestre, o estudante deverá aprender a planejar, executar e registrar testes manuais, identificar falhas e inconsistências, e compreender a importância da testabilidade e da qualidade no desenvolvimento de software desde as etapas iniciais.

Além disso, a disciplina visa familiarizar os estudantes com conceitos essenciais da automação de testes, tais como estruturas básicas de testes automatizados, uso de ferramentas introdutórias, métricas de cobertura e noções preliminares sobre pirâmide de testes. Espera-se que o aluno desenvolva a habilidade de analisar resultados de execução e interpretar relatórios simples para apoiar decisões de melhoria de código.

No campo da verificação, o curso introduz o raciocínio lógico necessário para formular e compreender propriedades de programas, utilizando pré e pós-condições, assertivas e noções introdutórias de triplas de Hoare. O foco está em ajudar o estudante a pensar de forma estruturada sobre o comportamento correto de um programa e sobre as condições necessárias para garantir sua validade.

Por fim, a disciplina também tem como objetivo estimular a comunicação técnica e o pensamento crítico, permitindo que o estudante justifique suas decisões de teste, discuta suas observações e reflita sobre a importância da validação contínua no ciclo de desenvolvimento. Espera-se que, ao terminar o curso, o aluno desenvolva autonomia inicial para investigar problemas, revisar seus próprios testes e compreender a relação entre testes, verificação e qualidade de software.

Conteúdo Programático**Semana:** 1**Título:** Introdução**Conteúdo:** Introdução a testes e verificação de software**Semana:** 2 a 7**Título:** Testes**Conteúdo:** Requisitos de Software

Regras de Negócio

Escrita de Testes

Pirâmide de Testes, Cobertura e Métricas de Qualidade

Automação de Testes - Laboratório

Definição Trabalho Área 1 - Início das Implementações

Semana: 8
Título: Apresentação de Trabalhos - Testes
Conteúdo: Apresentação de Trabalhos - Testes
Semana: 9 a 14
Título: Verificação
Conteúdo: Introdução à verificação - Lógica
Escrita de propriedades usando lógica
Assercoes no código - Lab
Triplas de Hoare
Triplas de Hoare - Invariantes
Definição Trabalho Verificação - Pré e Pós-Condições
Semana: 15
Título: Apresentação de Trabalhos - Verificação
Conteúdo: Apresentação de Trabalhos - Verificação
Semana: 16
Título: Recuperação
Conteúdo: Recuperação

Metodologia

A disciplina será desenvolvida em aulas teórico-práticas que combinam a apresentação de conceitos e técnicas com sua aplicação imediata pelos estudantes em exemplos, exercícios e trabalhos extraclasse. As 20 horas previstas para atividades teóricas e práticas contempladas neste Plano de Ensino incluem 12 encontros de 100 minutos cada (dois períodos de 50 minutos por encontro, um encontro semanal ao longo de 12 semanas), totalizando 1.200 minutos de atividades presenciais. Além disso, estão previstas 10 horas adicionais de atividades autônomas (600 minutos), realizadas sem contato direto com o professor, correspondentes a exercícios, estudos dirigidos e trabalhos extraclasse.

O professor poderá utilizar tanto aulas presenciais quanto atividades a distância, respeitando os limites estabelecidos pela UFRGS para disciplinas presenciais e empregando ferramentas apropriadas para atividades remotas. As atividades poderão contar com o apoio de professores assistentes (alunos de Pós-Graduação), especialmente em ações de natureza didática e de acompanhamento. Para tanto, poderão ser utilizados recursos como aulas síncronas ou assíncronas, gravações, demonstrações e demais tecnologias de apoio à aprendizagem.

O Moodle será utilizado como ambiente central de ensino, reunindo os planos de aula, materiais didáticos disponibilizados pelo professor, fóruns de comunicação, envio de tarefas, acompanhamento do desempenho e organização geral da disciplina.

Os estudantes devem seguir as regras definidas pelo professor responsável quanto ao uso de ferramentas de Inteligência Artificial (IA). As diretrizes relativas ao uso de IA podem variar entre os diferentes trabalhos e atividades, práticas ou teóricas, que compõem as experiências de aprendizagem e os critérios de avaliação da disciplina.

Experiências de Aprendizagem

A disciplina será conduzida por meio de uma combinação equilibrada de exposição teórica, atividades práticas em laboratório, exercícios orientados em sala de aula, tarefas autônomas e dois trabalhos aplicados. O objetivo é promover uma compreensão sólida dos fundamentos de testes e verificação, articulada ao desenvolvimento de habilidades práticas por meio de problemas reais de Engenharia de Software.

Os exercícios serão realizados individualmente ou em pequenos grupos, estimulando a colaboração, o diálogo técnico e a troca de perspectivas. As atividades autônomas complementam os temas apresentados em aula e ampliam a capacidade do estudante de aplicar os conceitos de forma independente.

No eixo de testes, as atividades focarão na análise de requisitos e regras de negócio, na escrita de cenários e casos de teste manuais e na implementação inicial de suítes de testes automatizados. No eixo de verificação, a prática envolverá a definição de pré e pós-condições, o uso de asserções no código e a especificação de invariantes simples, introduzindo os princípios fundamentais do raciocínio formal.

Critérios de avaliação

A avaliação da aprendizagem será composta por atividades autônomas, exercícios realizados em aula e um trabalho final dividido em duas partes. A seguir, apresentam-se os componentes avaliativos, seus pesos e as regras complementares.

1. Atividades Autônomas e Exercícios Dirigidos: 40%

Incluem atividades desenvolvidas em sala e tarefas realizadas individualmente, fora do horário de aula, ao longo do semestre.

2. Trabalho Final - 60%

O trabalho final será composto por duas partes, realizadas em grupo, com entregas parciais e finais via Moodle nas datas indicadas.

Parte 1 - 30%

Escrita de requisitos, regras de negócio e casos de teste

Parte 2 - 30%

Definição de pré e pós-condições;

Implementação de testes em Python, com cobertura de testes e cenários adequados.

As duas partes do trabalho serão apresentadas pelos grupos ao final do semestre.

3. Divulgação das Avaliações

A avaliação de cada trabalho será divulgada em até 3 semanas após sua realização ou 72 horas antes da prova de recuperação, prevalecendo o que ocorrer primeiro.

4. Cálculo do Conceito Final

O conceito final será obtido a partir da média ponderada das atividades autônomas (40%), Trabalho Final - Parte 1 (30%) e Trabalho Final - Parte 2 (30%), convertida em conceito conforme a escala:

Nota $\geq 9,0$; A

7,5 $\leq$ Nota < 9,0; B

6,0 $\leq$ Nota < 7,5; C

Nota < 6,0; D

5. Observações

A média geral dos estudantes será calculada somente para aqueles que obtiverem frequência mínima de 75% das aulas previstas.

Os estudantes que não atenderem a esse requisito receberão o conceito FF (Falta de Frequência).

Atividades de Recuperação Previstas

O aluno que obtiver conceito final D pode realizar uma prova de recuperação versando sobre todo o conteúdo da disciplina. Se a nota obtida nessa prova for igual ou superior a 6,0, o conceito mudará para C.

A recuperação final será realizada somente para os casos previstos na legislação: saúde, parto, serviço militar, convocação judicial, luto, etc., devidamente comprovados por processo aberto na Junta Médica da UFRGS ou no órgão competente, conforme o caso.

Tendo o direito à recuperação final, o professor estipulará a data, o horário e o local de sua realização.

Bibliografia

Básica Essencial

Ammann, P.; Offut, J.. Introduction to Software Testing. New York: Cambridge University Press, 2017. ISBN 9781107172012.

Pressman, Roger S.; Maxim, Bruce R.. Engenharia de Software: uma abordagem profissional. Porto Alegre: AMGH, 2021. ISBN 9786558040101.

Sommerville, Ian. Engenharia de software. São Paulo: Pearson Education, 2018.

Básica

Valente, Marco Tulio. Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade. online: Independente, 2020. Disponível em: <https://engsoftmoderna.info/>

Complementar

Sem bibliografias acrescentadas

Outras Referências
<i>Não existem outras referências para este plano de ensino.</i>

Observações
<i>Nenhuma observação incluída.</i>